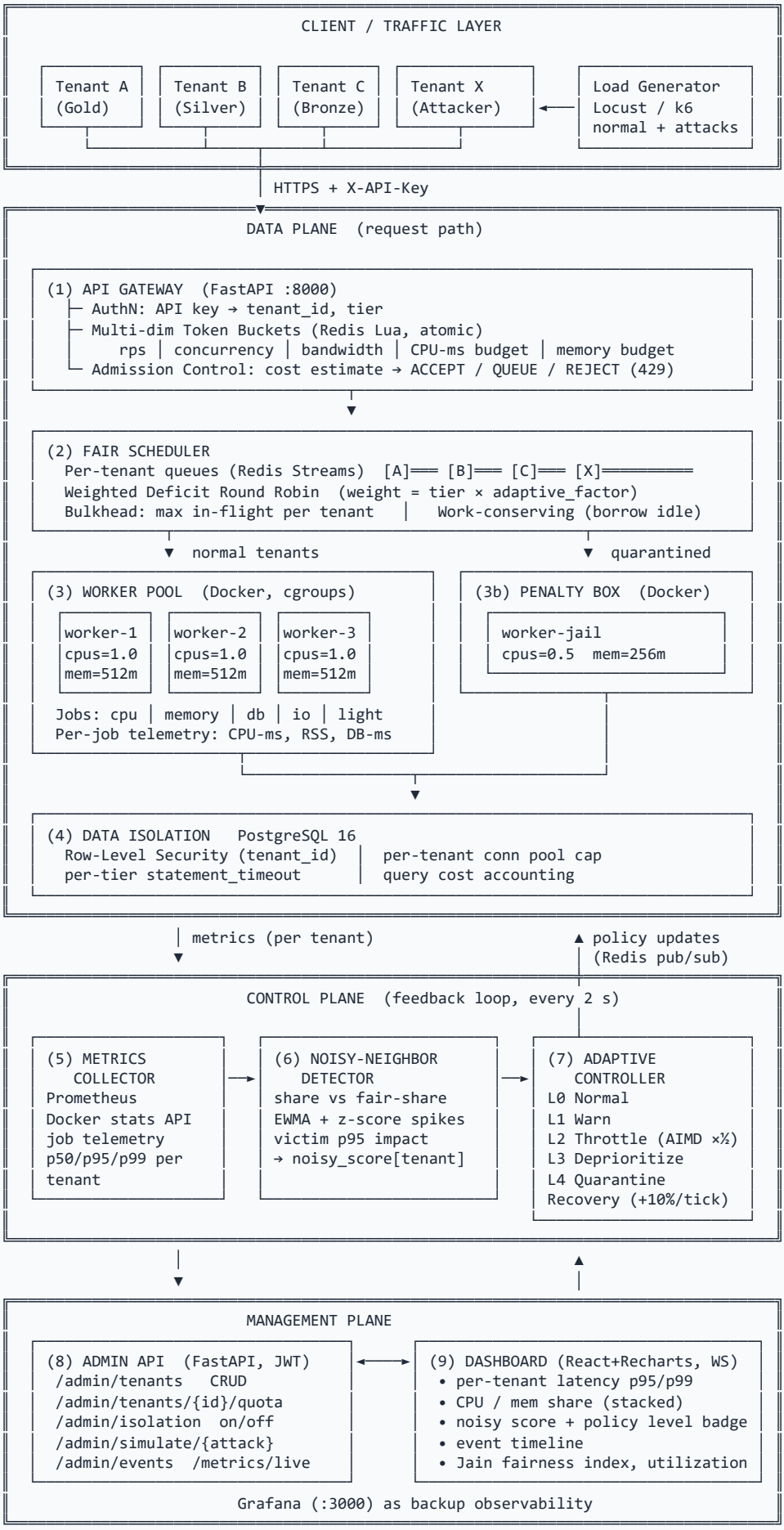
Multi-Tenant Isolation & Noisy-Neighbor Defense Platform

CodeAstra 2.0 · Cloud Computing & Distributed Systems · Problem Statement 3

Architecture | Tech Stack | Project Flow

"We built a mini cloud that gives every tenant a fair share, lets them burst when capacity is free, and automatically detects and contains abusive tenants in seconds, without wasting resources or killing anyone's service."

Version 1: Standard Technical Architecture


CLIENT / TRAFFIC LAYER
Tenant A (Gold)
Tenant B (Silver)
Tenant C (Bronze)
Tenant X (Attacker)
Load Generator Locust / k6 normal + attacks
HTTPS + X-API-Key
DATA PLANE (request path)
(1) API GATEWAY (FastAPI :8000)
AuthN: API key → tenant_id, tier
Multi-dim Token Buckets (Redis Lua, atomic)
rps | concurrency | bandwidth | CPU-ms budget | memory budget
Admission Control: cost estimate → ACCEPT / QUEUE / REJECT (429)
(2) FAIR SCHEDULER
Per-tenant queues (Redis Streams) [A]=== [B]=== [C]=== [X]==========
Weighted Deficit Round Robin (weight = tier × adaptive_factor)
Bulkhead: max in-flight per tenant | Work-conserving (borrow idle)
normal tenants
quarantined
(3) WORKER POOL (Docker, cgroups)
worker-1 cpus=1.0 mem=512m
worker-2 cpus=1.0 mem=512m
worker-3 cpus=1.0 mem=512m
Jobs: cpu | memory | db | io | light
Per-job telemetry: CPU-ms, RSS, DB-ms
(3b) PENALTY BOX (Docker)
worker-jail cpus=0.5 mem=256m
(4) DATA ISOLATION PostgreSQL 16
Row-Level Security (tenant_id)
per-tenant statement_timeout
per-tenant conn pool cap
query cost accounting
metrics (per tenant)
policy updates (Redis pub/sub)
CONTROL PLANE (feedback loop, every 2 s)
(5) METRICS COLLECTOR
Prometheus
Docker stats API
job telemetry
p50/p95/p99 per tenant
(6) NOISY-NEIGHBOR DETECTOR
share vs fair-share
EWMA + z-score spikes
victim p95 impact
→ noisy_score[tenant]
(7) ADAPTIVE CONTROLLER
L0 Normal
L1 Warn
L2 Throttle (AIMD ×¾)
L3 Deprioritize
L4 Quarantine
Recovery (+10%/tick)
MANAGEMENT PLANE
(8) ADMIN API (FastAPI, JWT)
/admin/tenants CRUD
/admin/tenants/{id}/quota
/admin/isolation on/off
/admin/simulate/{attack}
/admin/events /metrics/live
(9) DASHBOARD (React+Recharts, WS)
• per-tenant latency p95/p99
• CPU / mem share (stacked)
• noisy score + policy level badge
• event timeline
• Jain fairness index, utilization
Grafana (:3000) as backup observability

The 3 Planes

Plane	Purpose	Components
Data Plane	Runs tenant work in isolation	Gateway, Scheduler, Worker Pools, Penalty Box, PostgreSQL
Control Plane	Watches, decides and reacts automatically	Metrics Collector, Noisy-Neighbor Detector, Adaptive Controller
Management Plane	Lets humans see and control the system	Admin API, Dashboard, Grafana

The 5 Layers of Isolation (not just a rate limit)

Layer	Mechanism	Stops
L1 Request	Token buckets: rps, concurrency, bandwidth	Request floods
L2 Cost	CPU-ms / memory budgets charged after each job	"Few but heavy" requests
L3 Scheduler	Weighted fair queue + per-tenant bulkhead	Queue hogging, line cutting
L4 Kernel	Docker cgroups (cpus, memory, pids) + penalty-box container	CPU bombs, memory hogs
L5 Data	Postgres RLS + per-tenant connection pool + query timeout	DB floods, data leaks

Tech Stack

Concern	Choice	Why
Gateway / Scheduler / Admin API	Python 3.12 + FastAPI + asyncio	Fast to build, async, auto Swagger docs
Quota and rate state	Redis 7 + Lua scripts	Atomic token buckets, Streams for queues, pub/sub for policies
Database	PostgreSQL 16 (Row-Level Security)	Shows data isolation, not just compute isolation
Compute isolation	Docker Compose + cgroup limits	Real kernel-level isolation
Resource measurement	psutil, getrusage, Docker stats API	Per-job CPU-ms and RSS, per-container usage
Metrics	Prometheus + Grafana	Per-tenant latency histograms (p50/p95/p99)
Dashboard	React (Vite) + Recharts + Tailwind, WebSocket	Live "attack control room" UI
Attack generation	Locust (or k6)	Scriptable normal and abusive traffic
Auth	API keys (hashed) + JWT for admin	Simple and enough for a demo

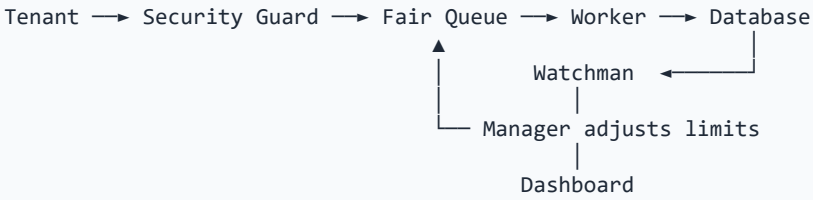
Adaptive Throttling Levels

Level	Trigger	Action
L0 Normal	score < 1.5	Full rate, can borrow idle capacity
L1 Warn	score ≥ 1.5 for 5 s	Event logged, dashboard alert
L2 Throttle	score ≥ 2	AIMD: rate × 0.5 each tick
L3 Deprioritize	still high	Scheduler weight reduced to 0.25
L4 Quarantine	score ≥ 4 or memory-limit hits	Jobs routed to the penalty-box container
Recovery	score < 1 for 15 s	+10% per tick, step down one level

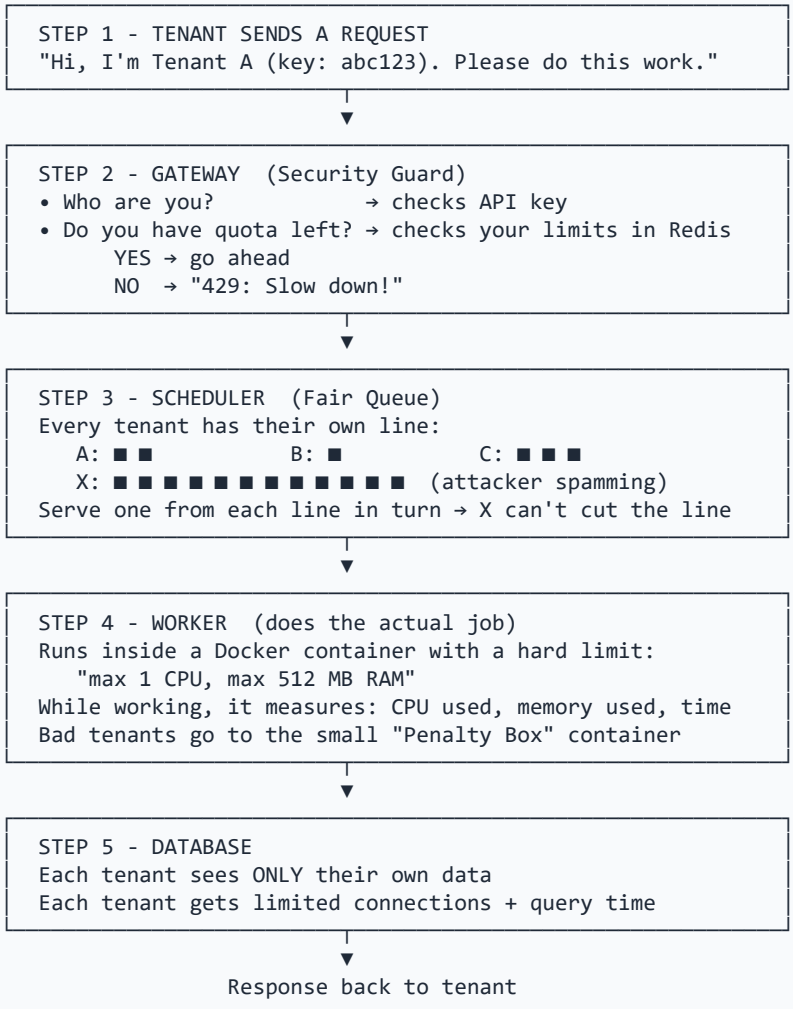
```
noisy_score[t] = max_r( share[t,r] / fair_share[t] ) × (1 + victim_impact)  (+ spike bonus if z > 3)
  → a tenant is flagged ONLY if it over-uses AND other tenants are actually suffering
```

Version 2: Simple Project Flow (Team View)

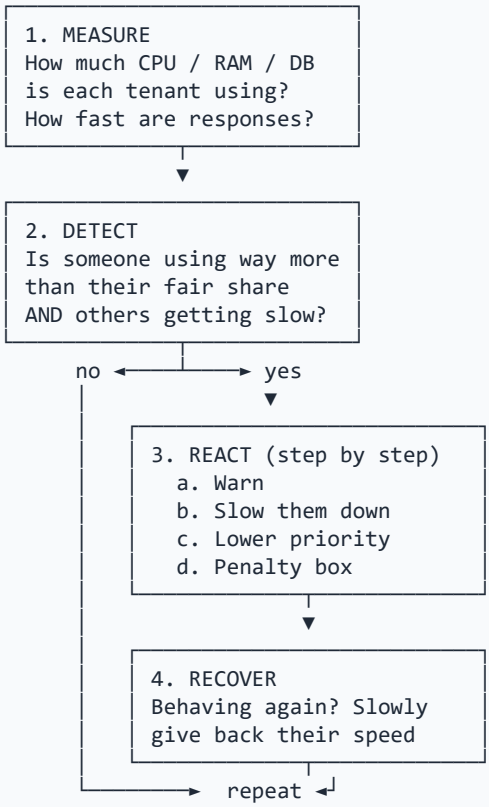
The Big Picture in One Line



A Request's Journey, Step by Step



The Automatic Protection Loop (runs every 2 seconds)



Who Builds Which Box

Member	Part	Builds
P1	Steps 1 + 2	Gateway, API keys, quotas, token buckets, admin API
P2	Steps 3 + 4 + 5	Fair queue, Docker workers, penalty box, Postgres isolation
P3	Protection Loop	Metrics collector, detector, adaptive controller
P4	Dashboard + Attacks	React dashboard, Locust attack scripts, demo results

The Demo in Pictures

Tenant	Act 1: Normal	Act 2: Attack, Protection OFF	Act 3: Attack, Protection ON
A	40 ms	**900 ms**	**45 ms**
B	42 ms	**850 ms**	**48 ms**
C	38 ms	**920 ms**	**41 ms**
X (attacker)	-	hogging everything	in penalty box (slow, but alive)

Latency figures are target/expected values; replace with measured results after the experiments.

Presentation tip: Put Version 1 on the architecture slide, then walk the judges through it using the Version 2 story ("let's follow one request..."). Judges remember stories better than boxes.